

Discussion Module:

DSE Search, Text Analytics - Part 2

Wrap up remainder of Text Analytics.

Not these, these are coming-

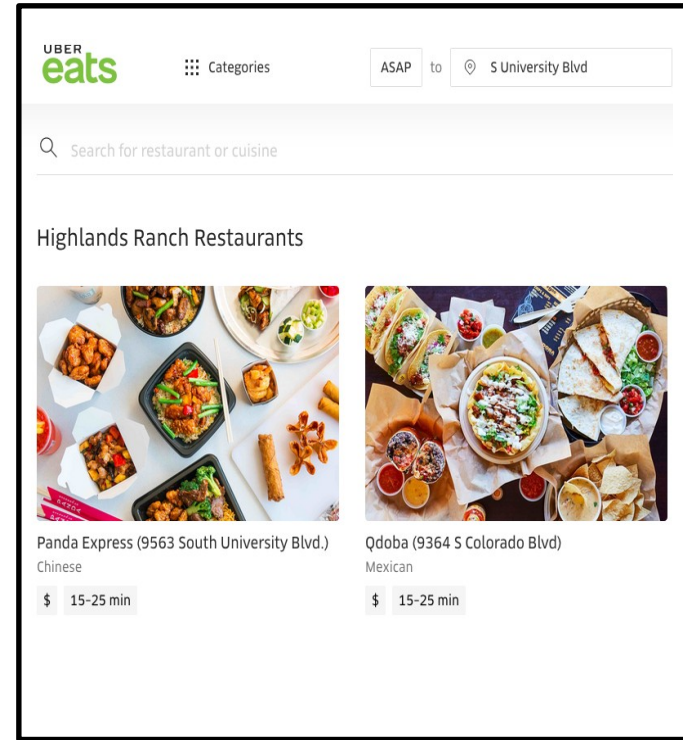
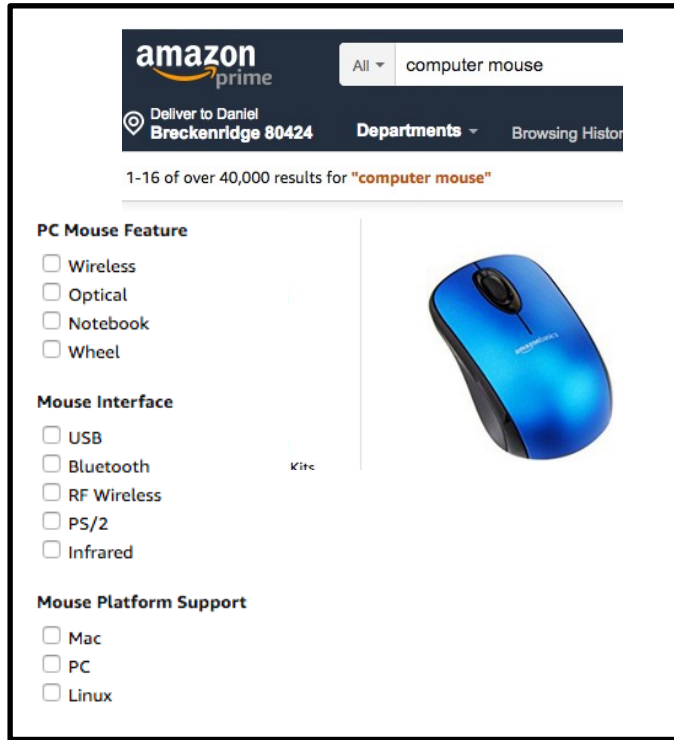
- A better treatment of query predicate, query functions, other.
- Geospatial/spatial (time series)
- Capacity planning, tuning

Discussion Lab:

What do you have, What do you need-



Amazon, Uber Eats: Name 3 You Have|Need



End of Discussion Lab:

DSE Search (Derived Fields)



(Derived fields)

- copyField
- dynamicField (pass until Geospatial)
- (Default Field)
- (Field Transformers)

copyField: Its own object, ALTER SEARCH INDEX .. ADD copyField..

Use cases:

- Need multiple treatments (different analyzers) for a given field (multiValued=false okay)
- Group multiple query attributes to a single location (ease of use, a default field)
 - multiValued = false // Attractive, but ..
 - multiValued = true // Required for M:1
- Other



Source fields not stored in index. (No source fields are stored in index.)

Destination field not stored in DSE table proper. (No key column data is stored in the table proper.)

End to End Example:

copyField



copyField example .. page 1 of 10



```
DROP KEYSPACE ks_7449;  
CREATE KEYSPACE ks_7449 WITH  
REPLICATION =  
    {'class': 'SimpleStrategy', 'replication_factor': 1};  
USE ks_7449;  
CREATE TABLE t6  
(  
    col1          TEXT,  
    col2          TEXT,  
    col3          TEXT,  
    col4          TEXT,  
    col5          TEXT,  
    col6          TEXT,  
    col7          TEXT,  
    col8          TEXT,  
    col9          TEXT,  
    col0          TEXT,  
    PRIMARY KEY ((col1, col2)) );
```

```
CREATE SEARCH INDEX ON t6 WITH  
COLUMNS col1,  
    * { excluded : true };
```

copyField example .. page 2 of 10



```
INSERT INTO t6  
  (col1, col2, col3, col4, col5,  
   col6, col7, col8, col9, col0)  
VALUES ('aaa', 'aaa', 'StarBucks', 'aaa', 'aaa',  
       'aaa', 'aaa', 'aaa', 'aaa', 'aaa');
```

```
VALUES ('bbb', 'bbb', 'bbb', 'Chuck', 'David',  
       'Todd', 'bbb', 'bbb', 'bbb', 'bbb');
```

```
VALUES ('ccc', 'ccc', 'ccc', 'Chuck', 'Mary',  
       'Lincoln', 'ccc', 'ccc', 'ccc', 'ccc');
```

```
VALUES ('ddd', 'ddd', 'ddd', 'David', 'Chuck',  
       'Todd', 'ddd', 'ddd', 'ddd', 'ddd');
```

copyField example .. page 3 of 10

```
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fields.field[@name='col3', @type='StrField',
    @indexed='false', @multiValued='false', @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fieldType[@name='TextField71',
    @class='solr.TextField'] WITH
    '{"analyzer":{"tokenizer":{"class":
      "solr.KeywordTokenizerFactory"},
      "filter":[{"class":"solr.LowerCaseFilterFactory"}]}}';
```

```
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fieldType[@name='TextField72',
    @class='solr.TextField'] WITH
    '{"analyzer":{"tokenizer":{"class":
      "solr.KeywordTokenizerFactory"},
      "filter":[{"class":"solr.BeiderMorseFilterFactory"}]}}';
```



indexed=false



**OLD
OLD
OLD**

copyField example .. page 4 of 10

```
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fields.field[@name='col71',
    @type='TextField71', @indexed='true', @multiValued='false',
    @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fields.field[@name='col72',
    @type='TextField72', @indexed='true', @multiValued='false',
    @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t6
  ADD copyField[@source='col3', @dest='col71'];
ALTER SEARCH INDEX SCHEMA ON t6
  ADD copyField[@source='col3', @dest='col72'];
```

```
RELOAD SEARCH INDEX ON t6;
REBUILD SEARCH INDEX ON t6;
```

**OLD
OLD
OLD**



copyField example .. page 5 of 10

```
SELECT col1, col3 FROM t6  
WHERE col1 = 'aaa' ;
```

```
// Fails, no index on col3  
SELECT col1, col3 FROM t6  
WHERE col3 = 'StarBucks';
```

```
// Fails, field not in table, needs solr_query predicate  
SELECT col1, col3 FROM t6  
WHERE col71 = 'StarBucks';
```

```
SELECT col1, col3 FROM t6  
WHERE solr_query = '{ "q" : "col71:starbucks"}';
```

```
SELECT col1, col3 FROM t6  
WHERE solr_query = '{ "q" : "col72:starbux"}';
```



why ?



copyField example .. page 6 of 10

OLD
OLD
OLD

```
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fields.field[@name='col4', @type='StrField',
    @indexed='false', @multiValued='false', @stored='false'];
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fields.field[@name='col5', @type='StrField',
    @indexed='false', @multiValued='false', @stored='false'];
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fields.field[@name='col6', @type='StrField',
    @indexed='false', @multiValued='false', @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fields.field[@name='col73', @type='StrField',
    @indexed='true', @multiValued='false', @stored=false'];
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fields.field[@name='col74', @type='StrField',
    @indexed='true', @multiValued='true', @stored='false'];
```



sortable ?

copyField example .. page 7 of 10



```
ALTER SEARCH INDEX SCHEMA ON t6 ADD copyField[@source='col4',  
  @dest='col73', @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t6 ADD copyField[@source='col5',  
  @dest='col73', @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t6 ADD copyField[@source='col6',  
  @dest='col73', @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t6 ADD copyField[@source='col4',  
  @dest='col74', @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t6 ADD copyField[@source='col5',  
  @dest='col74', @stored='false'];
```

```
ALTER SEARCH INDEX SCHEMA ON t6 ADD copyField[@source='col6',  
  @dest='col74', @stored='false'];
```

```
RELOAD SEARCH INDEX ON t6;
```

```
REBUILD SEARCH INDEX ON t6;
```

copyField example .. page 8 of 10

```
SELECT col1, col4, col5, col6 FROM t6
WHERE solr_query = '{"q" : "col73:(Chuck David Todd)"}';
```

col1	col4	col5	col6
ddd	David	Chuck	Todd
bbb	Chuck	David	Todd
ccc	Chuck	Mary	Lincoln

```
SELECT col1, col4, col5, col6 FROM t6
WHERE solr_query = '{"q" : "col74:(Chuck David Todd)"}';
```

col1	col4	col5	col6
ddd	David	Chuck	Todd
bbb	Chuck	David	Todd
ccc	Chuck	Mary	Lincoln



Hint: Could have a perfect 1.0 score

copyField example .. page 9 of 10

```
SELECT col1, col4, col5, col6 FROM t6  
WHERE solr_query = '{ "q" : "col73:(+Chuck +David +Todd)"}';  
SELECT col1, col4, col5, col6 FROM t6  
WHERE solr_query = '{ "q" : "col74:(+Chuck +David +Todd)"}';
```

col1	col4	col5	col6
ddd	David	Chuck	Todd
bbb	Chuck	David	Todd



why ?

copyField example .. page 10 of 10

// Finds no rows

```
SELECT col1, col4, col5, col6 FROM t6  
WHERE solr_query = 'col73:"Chuck David Todd";  
SELECT col1, col4, col5, col6 FROM t6  
WHERE solr_query = 'col74:"Chuck David Todd";
```



why ?

https://docs.datastax.com/en/dse/5.1/dse-dev/datastax_enterprise/search/queryTerms.html

https://docs.datastax.com/en/dse/5.1/dse-dev/datastax_enterprise/search/advancedTerms.html



End to End Example:

copyField, Review



dynamicField: Also its own object

Use cases:

- Geospatial/spatial, definitely
 - Minimally; take a Lat/Lon pair, split, and allow for a two dimensional index
 - Pass for now-
- Dynamic schemas ?? (Not with DSE)

Best practices

- Avoid or limit the use of dynamic fields.

Apache Lucene® allocates memory for each unique field (column) name. For example, for a row with columns A, B, C, and another row with B, D, E, Lucene allocates 5 chunks of memory. For millions of rows, the heap is unwieldy.

- Instead of using dynamic fields, use a default query field, and then perform queries against the combined field.
- Use the `FieldInputTransformer` (FIT) API.

Source: https://docs.datastax.com/en/dse/5.1/dse-dev/datastax_enterprise/search/usingDynamicFields.html

https://docs.datastax.com/en/dse/6.0/dse-dev/datastax_enterprise/search/unsupportedSearchFeatures.html



Default Field:

Use cases:

- Handy ?



edismax.uf ??

- Word proximity; some predicate expressions are hard without presence of a default field
- Config property, not schema property/object

End to End Example:

(Default Field), and Word
Proximity



Default Field Example .. page 1 of 5



```
DROP KEYSPACE IF EXISTS ks_7449;
```

```
CREATE KEYSPACE ks_7449 WITH REPLICATION =  
    {'class': 'SimpleStrategy', 'replication_factor': 1};
```

```
USE ks_7449;
```

```
CREATE TABLE t7  
(  
    col1          TEXT PRIMARY KEY,  
    col2          TEXT,  
    col3          TEXT  
);
```

Default Field Example .. page 2 of 5



```
INSERT INTO t7 (col1, col2, col3) VALUES
  ('aaa', 'test space', 'test');
INSERT INTO t7 (col1, col2, col3) VALUES
  ('bbb', 'space test', 'test');
INSERT INTO t7 (col1, col2, col3) VALUES
  ('ccc', 'test one two space', 'test');
INSERT INTO t7 (col1, col2, col3) VALUES
  ('ddd', 'test one two three space', 'test');
INSERT INTO t7 (col1, col2, col3) VALUES
  ('eee', 'test one two three four space', 'test');

CREATE SEARCH INDEX ON t7 WITH COLUMNS
  col1, * { excluded : true };
```

Default Field Example .. page 3 of 5

OLD
OLD
OLD

// KeywordTokenizer does not work here; need tokens

```
ALTER SEARCH INDEX SCHEMA ON t7
  ADD fieldType[@name='TextField81',
    @class='solr.TextField'] WITH
    '{"analyzer":{"tokenizer":
    {"class":"solr.StandardTokenizerFactory"}}}';
```



**Can we ever sort on col2 ?
What is the (default) sort ?**

```
ALTER SEARCH INDEX SCHEMA ON t7
  ADD fields.field[@name='col2', @type='TextField81',
    @default='true'];
```

```
RELOAD SEARCH INDEX ON t7;
REBUILD SEARCH INDEX ON t7;
```

Default Field Example .. page 4 of 5

```
SELECT * FROM t7 WHERE solr_query='col2:"test space"~10';
```

col1	col2	col3	solr_query
aaa	test space	test	null
bbb	space test	test	null
ccc	test one two space	test	null
ddd	test one two three space	test	null
eee	test one two three four space	test	null

```
SELECT * FROM t7 WHERE solr_query='col2:"test space"~2';
```

aaa	test space	test	null
bbb	space test	test	null
ccc	test one two space	test	null

```
SELECT * FROM t7 WHERE solr_query='col2:"test space"~1';
```

aaa	test space	test	null
-----	------------	------	------

Default Field Example .. page 5 of 5

```
// Fails with a,  
// InvalidRequest: Error from server: code=2200 [Invalid query]  
// message="no field name specified in query and no default specified  
SELECT * FROM t7 WHERE solr_query=""test space"~2';
```

```
ALTER SEARCH INDEX CONFIG ON t7 SET defaultQueryField = 'col2' ;
```

```
RELOAD SEARCH INDEX ON t7;  
REBUILD SEARCH INDEX ON t7;
```

```
SELECT * FROM t7 WHERE solr_query=""test space"~2';
```



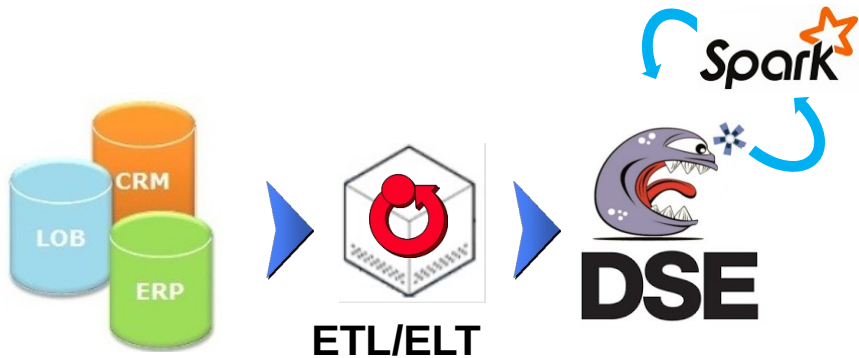
**Or, at least you know
why this predicate
works-**

End to End Example:

(Default Field), and Word
Proximity, Review



Field Transformers: Current landscape



Newly created data stored in index, not in DSE table columns proper.
But, still super handy.

[https://
www.datastax.com/dev/blog/dse-field-tran
sformers](https://www.datastax.com/dev/blog/dse-field-transformers)

- An eleven page article written using Java and Maven, this article walks you through the complete set of steps to design and implement.
- In effect, take a value, store it in a DSE table, and upon insert/update (and delete), apply a (custom parser), pick out the specific and distinct data elements, and DSE Search index them.
- There are only two Java classes you have to extend, FieldInputTransformer, and FieldOutputTransformer. Standard Java, and much of this Java code is boilerplate.

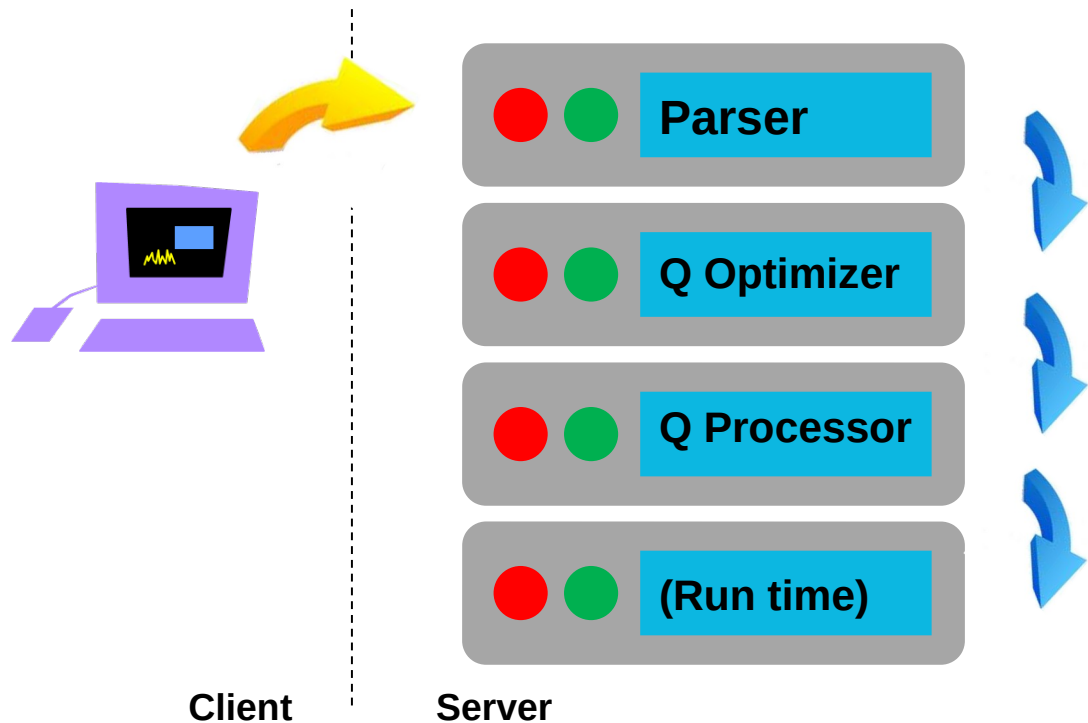
DSE Search Text Query Parsing, Scoring



- Text, score (relevancy)
- Geospatial, `geodist()`
- Spatial, `dist()`, other
- Scalars (none, sort)



Database Server Backends: Stages



**What do you know
right now ?**

**Why are parser and
(scoring) related ?**

DSE Search (Lucene) Scoring Model, Part 1 of 2



- tf, Term Frequency. Count per doc, higher == higher score
- idf, Inverse Doc Freq. Count of term across all docs. Denominator in equation $tf / (x)$, lower == higher score
- coord, Coordinator Factor. Higher count of distinct terms found in given doc == higher score
- fieldNorm, Field Length. More terms per doc == lower score

Source: <http://www.solrtutorial.com/solr-search-relevancy.html>

- Until Lucene 4, only Vector Space Model (Tf-Idf)
 - “Lucene Scoring Model”, the default; combination of VSM and Boolean (AND, OR)
-
- Post Lucene 4-
(Scoring Model is pluggable)
 - Okapi BM (not common)
 - Language Model (Which words are common/rare in this language)
 - Divergence from Randomness (fraud, security)
 - Information-based (Sentiment)

DSE Search (Lucene) Scoring Model, Part 2 of 2

- Scores not meaningful across queries
- Score normalized; always between 0 and 1
- Fast, hides complexity

Source: https://en.wikipedia.org/wiki/Vector_space_model

Pro:

- Simple model based on linear algebra
- Term weights not binary
- Allows computing a continuous degree of similarity between queries and documents
- Allows ranking documents according to their possible relevance
- Allows partial matching

Con:

- Long documents are poorly represented because they have poor similarity values (a small scalar product and a large dimensionality)
- Search keywords must precisely match document terms; word substrings might result in a "false positive match"
- "Semantic sensitivity; documents with similar context but different term vocabulary won't be associated, resulting in a "false negative match"
- The order in which the terms appear in the document is lost in the vector space representation
- Theoretically assumes terms are statistically independent
- Weighting is intuitive but not very formal

Score: Example, Part 1 of 2



```
"response": {  
  "numFound": 2,  
  "start": 0,  
  "maxScore": 0.7590632,  
  "docs": [  
    {  
      "col1": "ddd",  
      "col2": "test one two three space",  
      "score": 0.7590632  
    },  
    {  
      "col1": "eee",  
      "col2": "test one two three four space",  
      "score": 0.6454073  
    }  
  ]  
}
```

Solr Admin UI

- > Core
- > Query TAB
- > **fl = *, score**
- > **debugQuery=true**

Request-Handler (qt)

/select

— common —

q

col2:three

fq

sort

start, rows

0

fl

*,score

df

json.facet

Raw Query Parameters

key1=val1&key2=val2

wt

json

☐ indent

☒ debugQuery

Score: Example, Part 2 of 2

"ddd": "0.7590632 = weight(col2:three in 4) [], result of: 0.7590632 = score
(doc=4,freq=1.0 = termFreq=1.0), product of: 0.87546873 = idf(docFreq=2, docCount=5)
0.86703634 = tfNorm, computed from: 1.0 = termFreq=1.0 1.2 = parameter k1
0.75 = parameter b 3.8 = avgFieldLength 5.2244897 = fieldLength"

"eee": "0.6454073 = weight(col2:three in 0) [], result of: 0.6454073 = score
(doc=0,freq=1.0 = termFreq=1.0), product of: 0.87546873 = idf(docFreq=2, docCount=5)
0.73721343 = tfNorm, computed from: 1.0 = termFreq=1.0 1.2 = parameter k1
0.75 = parameter b 3.8 = avgFieldLength 7.111111 = fieldLength"

Solr Admin UI

- > Core
- > Query TAB
- > fl = *, score
- > debugQuery=true

Scoring: q and fq (fq, not covered yet)

- Most queries thus far (JSON) have been “q”, query predicate
- “fq” are a further set of predicates applied to the results of “q”
 - Not a correlated subquery or similar, (no new columns, rows)
 - Just further (filtering)
- fq filter activity does not affect score

- Index time, No (Solr, Yes)
- Query time, Yes
- Using boost, omitNorms=false
- (Not boost, save space, = true)

col4	col5	col5
David	Chuck	Chuck
Chuck	Mary	Mary
Chuck	David	David
aaa	aaa	aaa

```
SELECT * FROM t6
WHERE solr_query =
  '{ "q" : "(col73:%huck^8 OR
  col74:chuck~1)" }' ;
```

Boosting: Index and Query, Positive and Negative

- Caret; pos value , alter the score for the given predicate (Only Lucene supports neg boost; Solr and thus DSE doesn't)
- Tilde (unrelated to boost) How many character misspellings (Low numbers only.)

Remainder of Field Type, Field Attributes

Discussion Lab:

Matching pairs – Match the field type and field properties on the right with the correct term on the left

Field Type, Field, Attributes: Covered

Field Type

Name ☐

Class ☐



Field

☐ Name

☐ Type

Field Type, Field, Attributes: Covered

Word Proximity

☐

Boosting, false=Y

☐

Highlighting, false=N

☐

(Irrelevant, ignored)

☐

(Space savings)

☐

multiValued

☐☐

indexed=true

☐

indexed=false

☐

stored=t|f

☐

docValues=t|f

☐

multiValued=true

☐

multiValued=false

☐

default=str

☐

precisionStep=n

☐

positionIncrementGap=n

☐

termVectors=t|f

☐

termPositions=t|f

☐

termOffsets=t|f

☐

omitNorms=t|f

Field Type, Field, Attributes: Covered

copyField M:1 ☐

copyField 1:1 ☐

Collection ☐

Non-collection ☐

☐

indexed=true

☐

indexed=false

☐

stored=t|f

☐

docValues=t|f

☐

multiValued=true

☐

multiValued=false

☐

default=str

☐

precisionStep=n

☐

positionIncrementGap=n

☐

termVectors=t|f

☐

termPositions=t|f

☐

termOffsets=t|f

☐

omitNorms=t|f

Field Type, Field, Attributes: Covered

StrField ☐

TextField ☐

TrieDoubleField ☐



- ☐ indexed=true
- ☐ indexed=false
- ☐ stored=t|f
- ☐ docValues=t|f
- ☐ multiValued=true
- ☐ multiValued=false
- ☐ default=*str*
- ☐ precisionStep=n
- ☐ positionIncrementGap=n
- ☐ termVectors=t|f
- ☐ termPositions=t|f
- ☐ termOffsets=t|f
- ☐ omitNorms=t|f

Field Type, Field, Attributes: Covered

StrField ☐

TextField ☐

TrieDoubleField ☐

Word Proximity ☐

docValues ☐



☐ KeywordTokenizer

☐ StandardTokenizer

☐ docValues

End of Discussion Lab:

Field Type, Field, Attributes: Not (Yet) Covered

- `autoGeneratePhraseQueries`, For text fields. If true, Solr automatically generates phrase queries for adjacent terms. If false, terms must be enclosed in double-quotes to be treated as phrases.
- `enableGraphQueries`, (needs graph query parser, not common)
- `docValuesFormat`, (not common)
- `postingsFormat`, (not common)
- `sortMissingFirst` `sortMissingLast`, Control the placement of documents when a sort field is not present

Field Type, Field, Attributes: Not (Yet) Covered

- omitTermFreqAndPositions, (related to omitForms), If true, omits term frequency, positions, and payloads from postings for this field. This can be a performance boost for fields that don't require that information.
- omitPositions, (related to omitNorms), Similar to omitTermFreqAndPositions but preserves term frequency information.
- termPayloads. (related to other term*)
- useDocValuesAsStored, (related to stored, not relevant)
- large, (related to stored, not relevant)
- default (related to stored, not relevant)

Limits, Unsupported Features

Unsupported Features/Limits: C* <-> DSE <-> Solr

Apache Solr and Lucene limitations apply to DSE Search. For example:

- The 2 billion records per node limitation as described in [Lucene limitations](#).
- The 1024 maxBoolean clause limit in [SOLR-4586](#).
- Solr field name policy applies to the indexed field names:
 - Every field must have a name .
 - Field names must consist of alphanumeric or underscore characters only.
 - Fields cannot start with a digit.
 - Names with both leading and trailing underscores (for example, `_version_`) are reserved.

Note: Non-compliant field names are not supported from all components. Backward compatibility is not guaranteed.

- Limitations and known Apache Solr issues apply to DSE Search queries. For example: incorrect [SORT](#) results for tokenized text fields.

Source: https://docs.datastax.com/en/dse/6.0/dse-dev/datastax_enterprise/search/unsupportedSearchFeatures.html

Unsupported Features/Limits: C* <-> DSE <-> Solr

Unsupported Apache Cassandra features

These limitations apply to DSE Search:

- Column aliases are not supported in solr_query queries.
- Continuous paging.
- Static columns
- Counter columns
- Super columns
- Thrift-compatible tables with column comparators other than UTF-8 or ASCII.
- PER PARTITION clause is not supported for DSE Search solr_query queries.
- Indexing frozen maps is not supported. However, indexing frozen sets and lists of native and user-defined (tuple/UDT) element types is supported.
- Using DSE Search with newly created COMPACT STORAGE tables is deprecated.

Source: https://docs.datastax.com/en/dse/6.0/dse-dev/datastax_enterprise/search/unsupportedSearchFeatures.html

Unsupported Features/Limits: C* <-> DSE <-> Solr

These limitations apply to DSE Search:

- DSE Search does not support [Solr Managed Resources](#).
- Solr schema fields that are both dynamic and multiValued only for CQL-based search indexes.
- The deprecated replaceFields request parameters on document updates for CQL-based search indexes. Instead, use the [suggested procedure](#) for inserting/updating data.
- Block joins based on the Lucene BlockJoinQuery in search indexes and CQL tables.
- Schemaless mode.
- Partial schema updates through the REST API after search indexes are changed.
For example, to update individual fields of a schema using the REST API to add a new field to a schema, you must change the `schema.xml` file, upload it again, and reload the core (same for copy fields).
- `org.apache.solr.spelling.IndexBasedSpellChecker` and `org.apache.solr.spelling.FileBasedSpellChecker`
Instead use `org.apache.solr.spelling.DirectSolrSpellChecker` for spell checking.
- The `commitWithin` parameter.
- The SolrCloud `CloudSolrServer` feature of SolrJ for endpoint discovery and round-robin load balancing.

Source: https://docs.datastax.com/en/dse/6.0/dse-dev/datastax_enterprise/search/unsupportedSearchFeatures.html

Unsupported Features/Limits: C* <-> DSE <-> Solr

- The `commitWithin` parameter.
- The SolrCloud `CloudSolrServer` feature of SolrJ for endpoint discovery and round-robin load balancing.
- The DSE Search configurable `SolrFilterCache` does not support auto-warming.
- DSE Search does not support the duration Cassandra data type.
- `SELECT` statements with `DISTINCT` are not supported with `solr_query`.
- `UnInvertedFieldRealtime` field value cache.
- `GetReplicationHandler`: Store & Restore.
- `useDocValuesAsStored` in schema fields and as a query request parameter.
- Graph queriesSolr SQLStreaming aggregations.
- Data import handler.
- Tuple/UDT subfield sorting and faceting.
- The `dataDir` parameter in `solrconfig.xml`.

Source: https://docs.datastax.com/en/dse/6.0/dse-dev/datastax_enterprise/search/unsupportedSearchFeatures.html

Unsupported Features/Limits: C* <-> DSE <-> Solr

Other unsupported features

- JBOD mode.
- The Solr updatelog is not supported in DSE Search.
The commit log replaces the Solr updatelog. Consequently, features that require the updateLog are not supported. Instead of using **atomic updates**, partial document updates are available by running the update with CQL.
- CQL Solr queries do not support native functions or column aliases as selectors.
- RamDirectoryFactory or other non-persistent DirectoryFactory implementations.
- **Tuple and UDT** limitations apply.

Source: https://docs.datastax.com/en/dse/6.0/dse-dev/datastax_enterprise/search/unsupportedSearchFeatures.html



End of Module:



Additional Content:

Analyzer: index or query

```
ALTER SEARCH INDEX SCHEMA ON t6
  ADD fieldType[@name='TextFieldB1',
    @class='solr.TextField'] WITH
    {"analyzer":{"type" : "index", "tokenizer":
      {"class":"solr.KeywordTokenizerFactory"},
      "filter":[{"class":"solr.LowerCaseFilterFactory"}]}};
```

```
ALTER SEARCH INDEX SCHEMA ON t6
  ...
  {"analyzer":{"type" : "query", "tokenizer":
    {"class":"solr.KeywordTokenizerFactory"},
    ...
```

```
SELECT col1, col4 FROM t6
WHERE solr_query = '{ "q" : "col4:david"}';
```

```
col1 | col4
-----+-----
ddd | David
```

The analyzer attribute to set an analyzer as index-only, or query-only; the default is both.

Phonetic Matching

Phonetic matching algorithms may be used to encode tokens so that two different spellings that are pronounced similarly will match.

- For overviews of and comparisons between algorithms, see
http://en.wikipedia.org/wiki/Phonetic_algorithm
<http://ntz-develop.blogspot.com/2011/03/phonetic-algorithms.html>
- Algorithms this category are:
 - Beider-Morse Phonetic Matching (BMPM)
 - Daitch-Mokotoff Soundex
 - Double Metaphone, Metaphone
 - SoundexRefined Soundex
 - Caverphone
 - Kölner Phonetik a.k.a. Cologne Phonetic
 - NYSIIS